

Warehouse storage and retrieval optimization via clustering, dynamical systems modeling, and GPU-accelerated routing

Magnus Bengtsson, Jens Wittsten, and Jonas Waidringer

ABSTRACT. This paper introduces a warehouse optimization procedure aimed at enhancing the efficiency of product storage and retrieval. By representing product locations and order flows within a time-evolving graph structure, we employ unsupervised clustering to define and refine compact order regions, effectively reducing picking distances. We describe the procedure using a dynamic mathematical model formulated using tools from random dynamical systems theory, enabling a principled analysis of the system’s behavior over time even under random operational variations. For routing within this framework, we implement a parallelized Bellman-Ford algorithm, utilizing GPU acceleration to evaluate path segments efficiently. To address scalability challenges inherent in large routing graphs, we introduce a segmentation strategy that preserves performance while maintaining tractable memory requirements. Our results demonstrate significant improvements in both operational efficiency and computational feasibility for large-scale warehouse environments.

1. INTRODUCTION

The exponential growth of the e-commerce sector has necessitated significant advancements in warehouse management strategies. Increasing consumer demand for expedited order fulfillment has placed pressure on logistics operations, and major retailers like Amazon, H&M, and Walmart have made substantial investments in automation and AI-driven warehouse management systems (WMS) to enhance efficiency. One of the primary cost drivers in warehouse operations is order picking, accounting for approximately 60% of total warehouse expenses [24]. It directly impacts labor costs, order fulfillment speed, and overall supply chain performance. A well-optimized picking strategy minimizes travel distance while maintaining flexibility to accommodate variations in order composition over time. Since order-picking routes can be represented as graphs, where nodes correspond to product locations, determining an optimal route for each order is a combinatorial optimization problem. To further enhance efficiency, order clustering strategies based on product location and demand frequency have been proposed, allowing frequently co-ordered products to be relocated closer together, thereby reducing long-term travel distances [25].

Shortest path algorithms play a key role in optimizing order picking. While Dijkstra’s algorithm [15] is a well-known approach, its sequential nature limits its efficiency for large-scale problems. The Bellman-Ford algorithm [4, 17], in contrast, is better suited for parallel computing [20] and has been successfully implemented on GPUs to accelerate shortest path computations [7]. However, in real-world warehouse settings, order picking involves combinatorial optimization, where factorial growth in possible routes makes exhaustive evaluation

impractical. In fact, finding a shortest path for picking operations is a version of the traveling salesman problem (TSP). Since the TSP is NP-complete [11], approximate methods are often necessary in practice.

In addition, warehouse managers frequently encounter unpredictable demand patterns and inventory fluctuations, challenges that static optimization methods struggle to address effectively. Instead, stochastic modeling approaches have been shown to significantly enhance logistics decision-making, particularly regarding inventory positioning and order fulfillment [18]. This is exemplified in Amazon’s fulfillment centers where stochastic modeling is incorporated in their robotic picking algorithm to balance pick efficiency and order cycle times. While clustering in the traditional data science sense isn’t explicitly used, the algorithm conceptually groups orders based on various criteria to optimize efficiency [1].

Derived from the analysis above, this study proposes an advanced warehouse optimization framework that integrates graph databases, unsupervised clustering, and GPU-accelerated route optimization with the Bellman-Ford algorithm. Unlike conventional WMS that rely on static zone allocations, our approach continuously repositions products based on demand fluctuations. This research extends prior work in warehouse route optimization and clustering methodologies [31, 5] by employing a random dynamical systems (RDS) framework that provides a rigorous mathematical foundation for analyzing storage and retrieval efficiencies and modeling uncertainty and variability. By leveraging GPU-based computing, our system can handle large-scale warehouses and high order volumes more effectively.

The system employs clustering on two levels: (1) Order clustering, which reorganizes storage layouts to concentrate frequently co-ordered items and minimize long-term travel distance, and (2) Picking node clustering, which reduces computational complexity in route optimization while maintaining near-optimal results. These techniques are analyzed mathematically in Sections 2 and 3 and evaluated through numerical experiments in Sections 4 and . By continuously adapting storage configurations and applying combinatorial optimization strategies, this approach balances short-term efficiency gains with long-term structural improvements, ensuring that warehouse operations remain effective even amid fluctuating order patterns.

To describe the implementation of our system in full detail we develop a mathematical model found in Section 2. The approach can be summarized as follows:

Products in the warehouse are organized using a graph database, which allows for real-time tracking of their positions. This system ensures that the location of each product is always known, facilitating efficient management of stock levels and movement. Incoming orders are clustered based on their similarity and proximity, grouping orders that share common product types or are located near each other within the warehouse. This clustering is also taken into account for calculating picking routes, which minimizes the travel distance for picking multiple orders, thereby increasing efficiency.

For each cluster of orders, a cluster center is determined. This center represents the optimal location within the warehouse for storing products that are frequently included in the cluster’s orders. The calculation of the cluster center takes into account the positions of all products in the cluster and aims to minimize the overall distance required for picking. When a product position is emptied during the picking process, the system triggers a reorganization of the warehouse. The product is restocked at a new position that is closer to the relevant cluster center, ensuring that products are always stored in positions that reduce the travel distance for future picking operations.

1.1. Related work. A central principle in classical storage location assignment problem (SLAP) is the *ABC classification* of inventory items according to turnover or demand frequency. In this approach, products are divided into a small number of categories—typically three—such that high-demand (A) items are placed in easily accessible locations, while medium- and low-demand (B and C) items occupy progressively less strategic zones. This strategy has long been recognized as effective for improving picker travel efficiency in picker-to-parts systems [25].

In our study, we adopt $K = 3$ clusters to reflect this well-established ABC structure. The intent is not to optimize the number of clusters per se, but to model how a warehouse that follows an ABC-type policy can reorganize its inventory dynamically as demand patterns evolve. The clustering thus serves as an operational analog of ABC zoning under dynamic conditions rather than as a data-driven search for an optimal K . For this purpose, we employ the K -means algorithm due to its simplicity, interpretability, and computational efficiency. More sophisticated clustering methods (e.g., density-based or hierarchical approaches) could in principle be used, but the aim here is to demonstrate the feasibility and benefits of cluster-driven reorganization rather than to compare clustering techniques. Future work may consider alternative clustering methods within the same framework to explore their potential impact on convergence speed and stability.

Classical research on order picking, as reviewed by de Koster et al. [12], highlights the importance of labor and travel-time considerations in warehouse performance. In contrast, the present work abstracts from such operational constraints—including labor assignment, congestion, and time scheduling—to focus on the structural dynamics of storage-location assignment and product reorganization. Our model assumes unrestricted relocation and omits explicit labor capacity limits, allowing us to isolate and analyze the structural effects of clustering on order consolidation and route optimization. Extending the framework to incorporate labor constraints or relocation delays would represent a natural direction for future research.

While classical studies of picker-to-parts systems emphasize operational batching—where multiple customer orders are grouped to minimize picker travel distance [12]—our approach performs a structural analog of this process. By clustering orders and their associated article types based on spatial and demand similarity, the model effectively performs a dynamic, data-driven form of batching at the storage-location level. This allows reorganization of stock positions that facilitates future co-picking, extending the traditional batching concept from route planning to layout adaptation.

Recent research has also highlighted a growing interest in adaptive storage assignment policies. Rizqi and Chou [27] integrate a mathematical optimization model with discrete-event simulation, using the former to propose storage and picking strategies and the latter to evaluate performance under realistic warehouse dynamics. Other approaches have focused on reinforcement learning: Teck et al. [33] employ deep reinforcement learning (proximal policy optimization, PPO) to optimize rack storage and routing in parts-to-picker systems, while Yerlikaya and Arıkan [35] introduce a multi-criteria class-based storage framework that adapts dynamically to order profiles. In contrast to these simulation- and RL-based frameworks, our contribution provides a tractable mathematical model for picker-to-parts systems that incorporates dynamic product reorganization through clustering. Performance is validated through large-scale numerical experiments supported by GPU-based computation, demonstrating the scalability and stability of the approach.

From a computational perspective, our GPU implementation of the routing component uses the Bellman–Ford algorithm for single-source shortest-path computation (SSSP). While

alternative GPU-oriented algorithms such as Δ -stepping can achieve higher asymptotic performance through fine-grained load balancing and reduced synchronization overhead [7], Bellman–Ford offers superior simplicity and generality. Its straightforward parallelization in CUDA enables transparent and reproducible implementation without reliance on external graph-processing libraries. Since routing constitutes only one step within the integrated clustering–RDS framework, Bellman–Ford provides an appropriate and stable baseline, with more specialized GPU SSSP methods left for future large-scale extensions.

Finally, the broader evolution of warehouse research toward data-driven and cyber-physical systems has been comprehensively reviewed by Boysen et al. [6]. They highlight how digitalization, sensor data, and real-time analytics enable adaptive storage assignment and control strategies, often under the umbrella of “Warehousing 4.0.” Our framework aligns with this development by integrating clustering-based reorganization and GPU-accelerated routing into a mathematically transparent model, bridging traditional optimization formulations with scalable, data-oriented computation.

2. MATHEMATICAL MODEL

We describe our warehouse optimization method and our numerical experiments using dynamical systems theory. We first introduce the state space (describing the different states of the warehouse) and give the definition of orders.

2.1. The state space. We consider a situation where N types of articles $a_1, a_2, \dots, a_N$, are stored in a warehouse at nodes arranged in three-dimensional space representing racks with shelves at different heights. Each shelf on a rack constitutes its own node, and each node is identified with an element in a three-dimensional array having integer coordinates

$$(i, j, k) \in [1, n_x] \times [1, n_y] \times [1, n_z] \subset \mathbb{N}^3, \quad (1)$$

where $\mathbb{N} = \{0, 1, 2, \dots\}$ is the set of natural number, and $n_x \times n_y \times n_z$ is the total number of nodes in the warehouse. We assume there is at most one article type a_n at each node (some nodes being empty), and identify each article type a_n with the corresponding ordinal $n \in [1, N]$. Empty nodes are assigned article type 0.

For each article type $n \in [1, N]$, let M_n be the maximum number of parcels that can be stored at an individual node. We assume this number is independent of which node is being considered; it only depends on what type of article is stored at that node. Introduce the space $\mathcal{A} = [0, N]^{n_x \times n_y \times n_z}$ of three-dimensional arrays. Each array $A = (a_{ijk})_{ijk} \in \mathcal{A}$ corresponds to one particular choice of how to distribute article types in the warehouse. (The array with only zeros represents an empty warehouse with no article types, and the array with only ones is a warehouse that only contains articles of type a_1 .) Each array element a_{ijk} is thus an integer n between 0 and N , representing the fact that the node at (i, j, k) contains article type a_n .

In other words, for each fixed $A = (a_{ijk})_{ijk} \in \mathcal{A}$, the node (i, j, k) contains articles of type $a_{ijk} \in [0, N]$. As described above, the maximum number of such articles at one node is $M_{a_{ijk}}$. For each $A = (a_{ijk})_{ijk} \in \mathcal{A}$ we now consider the fiber over A ,

$$F_A \mathcal{A} = \{M : M = (m_{ijk})_{ijk}, m_{ijk} \in [0, M_{a_{ijk}}]\}.$$

This describes all the possible inventory balances M (number of parcels of each article type) for a given article type distribution A . The state space is then the fibration defined by the

(disjoint) union

$$X = \bigcup_{A \in \mathcal{A}} \{A\} \times F_A \mathcal{A} = \{(A, M) : A \in \mathcal{A}, M \in F_A \mathcal{A}\},$$

where A varies over all the possible article type distributions. States in X will be denoted by x , so that $x = (A, M) \in X$. We may of course also represent X by attaching to each article type the corresponding inventory balance, so that

$$X \cong \{(a_{ijk}, m_{ijk})_{ijk} : a_{ijk} \in [0, N], m_{ijk} \in [0, M_{a_{ijk}}]\},$$

and we will not distinguish between these two representations.

Remark. In the description above we assume that nodes have integer coordinates and only allow one article type per node. This is a deliberate simplification made for clarity of presentation and ease of interpretation. At the cost of additional bookkeeping, the model can be generalized in two natural ways: to allow nodes with real-valued coordinates, and to allow multiple article types per node (commonly referred to as stock keeping units, or SKUs, in the warehouse literature).

First, multiple SKUs may be assigned to the same node by introducing an additional shelf index l , so that

$$(i, j, k, l) \in [1, n_x] \times [1, n_y] \times [1, n_z] \times [1, n_w] \subset \mathbb{N}^4,$$

where n_w denotes shelf capacity. Allocation then corresponds to a standard bin-packing problem, or to a rectangle-packing problem if shelves are treated as two-dimensional footprints [9, 34, 19, 26].

Second, node positions may be generalized to real coordinates by introducing a mapping

$$f_{\text{coord}} : \mathbb{N}^3 \rightarrow \mathbb{R}^3, \quad (i, j, k) \mapsto (x(i, j, k), y(i, j, k), z(i, j, k)),$$

so that route optimization is performed on the resulting continuous layout rather than the integer lattice in (1).

While these extensions would make the model more realistic, they are beyond the scope of this paper. Our focus here is on the clustering of orders and picking nodes, and the simplified setting suffices for this purpose. More general formulations, such as multi-SKU nodes or continuous coordinates, are natural directions for future work.

2.2. Orders. An order o is a vector

$$o = (m_1, \dots, m_N) \in \mathbb{N}^N$$

where $m_n \in \mathbb{N}$ is the number of parcels of article a_n contained in the order. (Here we shall by an order mean any order—individual customer orders or a collection of customer orders—that has been decided should be picked from the warehouse.) Given a state $x = (A, M)$ and an order o , the Order Processing algorithm (defined in Algorithm 1 in A below) and a route optimization algorithm (which involves the proposed clustering step based on similarity and proximity, see §2.4) provide the nodes from which the order should be collected, along with the picking route between these nodes. The first part is described by a map w given by

$$w : (x, o) \mapsto w(x, o) = (w_{ijk})_{ijk} \in \mathbb{N}^{n_x \times n_y \times n_z}, \quad (2)$$

where w_{ijk} is the number of parcels that should be collected from node (i, j, k) , and the total number of parcels in the order is $\sum_{i,j,k} w_{ijk} = m_1 + \dots + m_N$. The Update Stock algorithm (Algorithm 2) then calculates the intermediate state

$$(A, M') = (A, M - w(x, o)). \quad (3)$$

The Check Stock algorithm (Algorithm 3) verifies if there is sufficient stock to fill the order without picking the last parcel on a shelf, i.e., whether $m_{ijk} > w_{ijk}$ for all (i, j, k) . If this is not the case, i.e., if $m_{ijk} \leq w_{ijk}$ for some (i, j, k) such that $w_{ijk} \geq 1$, then the Check Stock and Move Articles algorithm of the warehouse optimization routine (Algorithm 4) computes a new warehouse location (i', j', k') at which to restock article a_{ijk} (collecting from the new location after emptying the balance at the old location), and the article array A is updated to A' in which a_{ijk} is replaced by 0, and $a_{i'j'k'}$ is changed from 0 to a_{ijk} . This gives an updated state

$$x' = f(x, o) := (A', M'), \quad (4)$$

which is the new state of the warehouse after having completed picking of the order o . If no nodes become empty when filling the order then the intermediate state (3) is taken as the new state x' in (4).

2.3. Random dynamical systems. To capture the stochastic evolution of warehouse states under repeated picking and resorting, we formulate the system as a random dynamical system (RDS), which provides a compact mathematical framework for modeling how random demand patterns drive successive reorganizations of stored articles over time. Standard mathematical references are Arnold [3] and Strogatz [32].

We use discrete time $n \in \mathbb{N}$, where $\mathbb{N} = \{0, 1, 2, \dots\}$ as before is the set of natural numbers. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. We take Ω to be the shift space consisting of sequences $\omega = (o_1, o_2, \dots)$ of orders o_n , and let $\theta : \Omega \rightarrow \Omega$ to be the shift operator $\theta\omega = (o_2, o_3, \dots)$. This models a sequence of orders $\omega \in \Omega$, placed with probability $\mathbb{P}$, coming in to the warehouse to be filled one after the other, with θ driving the flow of orders. Let X be the state space describing different states of the warehouse. Let $F : \omega \mapsto F_\omega$ be a random variable from Ω to the set of maps on X , i.e., for all $\omega \in \Omega$, F_ω is a map $F_\omega : X \rightarrow X$. Consider the *skew product* defined by $\Theta(\omega, x) = (\theta\omega, F_\omega(x))$. Iterating Θ gives

$$\Theta^n(\omega, x) = (\theta^n\omega, F_{\theta^{n-1}\omega} \circ F_{\theta^{n-2}\omega} \circ \dots \circ F_\omega(x))$$

for $n \geq 1$, where $\circ$ denotes composition of functions. The second coordinate is a *random dynamical system over θ* .

We let $f(x, o)$ be the function in (4) describing how the state x evolves to $x' = f(x, o)$ by filling the order o once, then, if necessary, restocking articles at new locations as determined by the optimization routine. With $\omega = (o_1, o_2, \dots)$ we then let $F_\omega(x) = f(x, o_1)$, so that $F_{\theta^{n-1}\omega}(x) = f(x, o_n)$ for $n \geq 1$. The simplest example is when $\omega = (o, o, \dots)$, corresponding to the same order o being placed over and over (see Experiment 1 in §4 below). To account for small differences in individual orders we will also consider the random example when $\omega = (o_1, o_2, \dots)$ and the orders o_n are independent, identically distributed perturbations of some fixed order o (see Experiments 2 and 3 in §4). In this case $\Theta^n(\omega, x_0)$ describes how the (initial) warehouse state x_0 evolves to

$$x_n = F_{\theta^{n-1}\omega} \circ \dots \circ F_\omega(x_0) = f(\bullet, o_n) \circ \dots \circ f(\bullet, o_1)(x_0) \quad (5)$$

by filling the first n orders in ω , having potentially restocked articles after each order at new locations as determined by the optimization routine. $\Theta^n(\omega, x_0)$ also contains the information that the next order to be filled is the first entry in $\theta^n\omega = (o_{n+1}, o_{n+2}, \dots)$, that is, o_{n+1} . We also note that in this case, the random difference equation

$$x_n = F_{\theta^{n-1}\omega}(x_{n-1}), \quad n \geq 1, \quad x_0 \in X,$$

generates a Markov chain $(x_n)_n$ with transitional probability $P(x, B) = \mathbb{P}\{\omega : F_\omega(x) \in B\}$, cf. [23]. Iteration of the picking and restocking process for a sequence of orders $o_1, o_2, \dots$ in this way is handled by the Main Order Processing algorithm (Algorithm 5).

2.4. Routes and clusters. Given a state x and an order o , a route is an ordering of the elements in the set of picking nodes

$$S_{\text{pick}} = S_{\text{pick}}(x, o) = \{(i, j, k) : w_{ijk} \neq 0\}$$

determined by (2). Routes are determined so that picking is completed from nodes on the same rack first (i.e., nodes that have the same (i, j) -coordinates) before moving on to the next rack. The stops along the route can then be described as a given ordering of the elements in the set of stopping nodes

$$S_{\text{stop}} = S_{\text{stop}}(x, o) = \{(i, j) : \text{there exists } k \text{ such that } (i, j, k) \in S_{\text{pick}}\},$$

and the number of stops is

$$n = \#S_{\text{stop}}. \quad (6)$$

At stopping nodes with more than one picking node, we can pre-determine the order in which picking is carried out—e.g., top to bottom—so this does not have to be included in the route optimization. We will therefore not distinguish between picking routes and routes between stopping nodes.

As explained in the introduction, we propose to group orders into clusters based on similarity and proximity to reduce computational complexity while still optimizing picking. The separation and concentration of the clusters is improved as time progresses by taking clustering into account when restocking. The algorithm restocks an article at an empty picking node whose (x, y) -coordinate (i.e., stopping node) is closest to the cluster center of the corresponding cluster. This reduces the area over which each cluster is spread out. Figure 1 shows an example of this phenomenon, which we will investigate further in Section 4 by performing several numerical experiments. Each of the top panels in Figure 1 show one of three clusters resulting from applying K -means clustering on a picking order o consisting of 200 article types, given a randomized initial warehouse state x_0 . The nodes in the warehouse have coordinates (i, j, k) with $i, j, k = 1, \dots, 10$. The stopping nodes in each cluster are indicated with color coding describing how many picking nodes there are at each stopping node. The three clusters are all more or less spread out over the entire square. In contrast, the bottom panels show K -means clustering when the same order o is going to be picked for the 100:th time; the warehouse state is then x_{99} with x_n as in (5), obtained after applying the resorting algorithm having already picked this order 99 times. Article types have now been restocked at new positions that are closer to the relevant cluster center, and clusters are clearly distinguishable and well separated.

To make this mathematically precise we introduce the following definition. Let m be the number of clusters. Given a state x and an order o , a cluster is a subset of S_{pick} which we will denote by

$$S_{\text{pick}}(\ell) = S_{\text{pick}}(\ell; x, o) \subset S_{\text{pick}}, \quad 1 \leq \ell \leq m,$$

such that the $S_{\text{pick}}(\ell)$ are mutually disjoint for $1 \leq \ell \leq m$ and

$$S_{\text{pick}} = \bigcup_{\ell=1}^m S_{\text{pick}}(\ell).$$

The stops in cluster ℓ are

$$S_{\text{stop}}(\ell) = S_{\text{stop}}(\ell; x, o) = \{(i, j) : \text{there exists } k \text{ such that } (i, j, k) \in S_{\text{pick}}(\ell)\}. \quad (7)$$

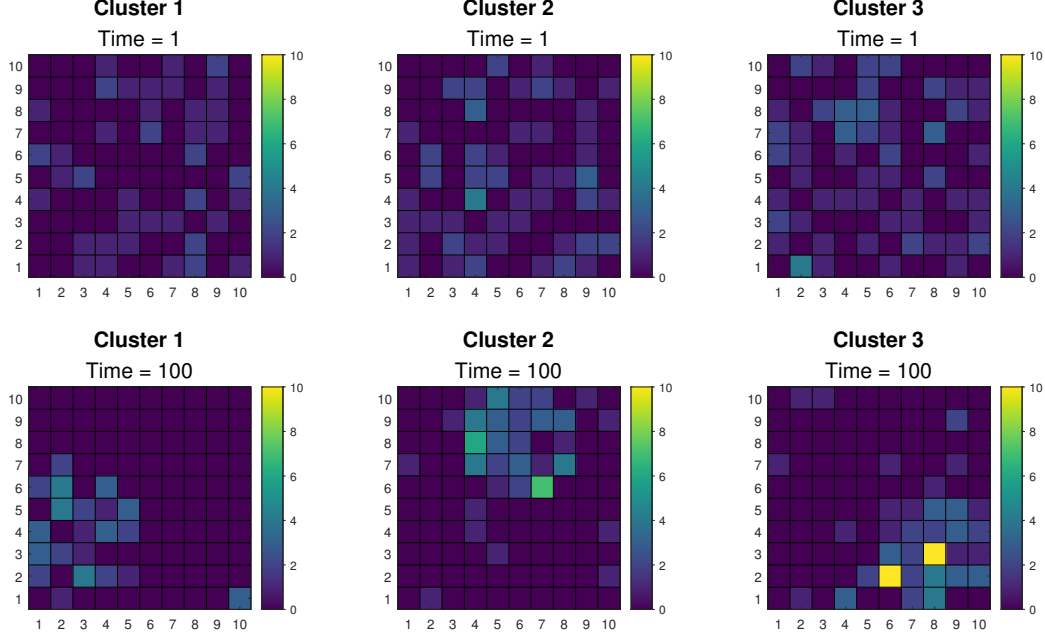


FIGURE 1. Each of the top panels show one of three clusters resulting from K -means clustering applied to an order with 200 article types given a randomized initial warehouse state x_0 . The nodes in the warehouse have coordinates (i, j, k) with $i, j, k = 1, \dots, 10$. Stopping nodes are indicated with color coding describing how many picking nodes a given stopping node contains. Each of the bottom panels show one of three clusters resulting from K -means clustering applied to the same order being picked for the 100:th time, that is, given the warehouse state x_{99} as in (5), obtained after 99 picking iterations. In the top panels, all three clusters are spread out over the entire warehouse with much overlap between clusters. In contrast, the clusters in the bottom panels are concentrated and much better separated.

Note that the sets $S_{\text{stop}}(1), \dots, S_{\text{stop}}(m)$ are not mutually disjoint in general, which means that a rack in the warehouse can belong to more than one cluster. In (7), the number of k 's such that $(i, j, k) \in S_{\text{pick}}(\ell)$ is the picking frequency at stop (i, j) for cluster ℓ . By taking picking frequency into account, clusters can be represented in two dimensions as in Figure 1, where the frequency is represented by the color coding of each stopping node.

We remark that we define clusters mathematically as a subset of the picking nodes for a given order, as opposed to dividing the picking nodes from a collection of orders into different groups or clusters. But since an order here can refer to the total collection of individual customer orders to be picked at a certain time our definition covers also the second scenario.

Because we will study routes between stopping nodes, we define cluster centers as the expected value of the location of stopping nodes in each cluster, taking picking frequency

into account. The cluster centers are then given for cluster ℓ by $(\bar{x}(\ell), \bar{y}(\ell))$ where

$$\bar{x}(\ell) = \frac{\sum_{(i,j,k) \in S_{\text{pick}}(\ell)} i}{\#S_{\text{pick}}(\ell)}, \quad \bar{y}(\ell) = \frac{\sum_{(i,j,k) \in S_{\text{pick}}(\ell)} j}{\#S_{\text{pick}}(\ell)},$$

with covariance matrix

$$\Sigma(\ell) = \begin{pmatrix} \sigma_{xx}^2(\ell) & \sigma_{xy}^2(\ell) \\ \sigma_{xy}^2(\ell) & \sigma_{yy}^2(\ell) \end{pmatrix}. \quad (8)$$

Here the variances and covariance are given by

$$\begin{aligned} \sigma_{xx}^2(\ell) &= \frac{\sum_{(i,j,k) \in S_{\text{pick}}(\ell)} (i - \bar{x}(\ell))^2}{\#S_{\text{pick}}(\ell)}, \\ \sigma_{yy}^2(\ell) &= \frac{\sum_{(i,j,k) \in S_{\text{pick}}(\ell)} (j - \bar{y}(\ell))^2}{\#S_{\text{pick}}(\ell)}, \\ \sigma_{xy}^2(\ell) &= \frac{\sum_{(i,j,k) \in S_{\text{pick}}(\ell)} (i - \bar{x}(\ell))(j - \bar{y}(\ell))}{\#S_{\text{pick}}(\ell)}. \end{aligned}$$

To study how clusters evolve as we apply our resorting procedure to a sequence of orders $o_1, o_2, \dots$ we let $\Sigma_n(\ell)$ be the covariance matrix of cluster ℓ as in (8) defined for a state x_{n-1} and order o_n , $n \geq 1$. Here x_n is given by (5), so that x_n is the warehouse state obtained after filling the first n orders in the sequence $o_1, o_2, \dots$. This will be used in Section 4 where we illustrate the time evolution of the mean and variance of clusters by plotting bivariate Gaussian probability density functions with corresponding mean and covariance matrices as n increases, see Figures 3, 4 and 5.

Remark. In the formulation above, relocations are assumed to be unrestricted, allowing any article to move freely between storage positions at each iteration. This simplifying assumption isolates the structural effects of clustering and routing from operational factors such as labour capacity or relocation delays, which are discussed as possible extensions in Section 6.

3. ROUTE OPTIMIZATION

In addition to clustering orders for reorganizing storage layouts, this paper presents a procedure for optimizing order picking routes by using clustering also within a given picking order. In the language of our mathematical model, we recall that a picking route is a route with total number of stops in the xy -plane given by n in (6). If there are more than one picking node at a picking stop (cf. §2.4) we pre-define the picking order for such nodes so this does not have to be included when calculating the route. However, we recall that the frequency of picking at each stop does matter, and that the restocking routine biases toward increased frequency. Over time (i.e., upon iteration of the resorting algorithm) this leads clusters to become more concentrated with smaller area and fewer stops, which in turn leads to a reduction in both travel distance and computational complexity. However, finding an optimal route can be a daunting task even after the number of stops have been reduced in this way, so in this section we discuss a technique to reduce computational complexity while maintaining near optimal results when calculating the route at each iteration of the resorting algorithm. We showcase the technique at the end of Section 4.

For large orders, where the computational complexity of route optimization can be $O(n!)$, we propose to use a GPU-based implementation of the Bellman-Ford algorithm to compute the route. This approach allows for efficient handling of large datasets by significantly reducing computational time. For clarity of exposition we will assume that the entrance

and exit are the same in the warehouse, and that these are not part of the route. Routes will then not be closed curves, and we will treat two routes as identical if they differ only in their travel direction. The number of possible routes between n picking stops (i.e., stopping nodes) is then $\frac{1}{2}n!$. To reduce computing costs we propose to use unsupervised machine learning to divide the picking stops into m clusters, then finding optimal routes in each cluster, and finally patching these routes together. This has the possibility of substantially reducing the computational complexity. For simplicity we begin with a naive discussion that doesn't leverage any other sophisticated algorithms to reduce complexity.

Theorem. *Let n be the number of picking stops, and let these be divided into m pairwise disjoint clusters, each containing $n_1, \dots, n_m$ picking stops, where $n_1 + \dots + n_m = n$. If an undirected non-closed route has been determined for each cluster then the total number of possible global undirected routes connecting them is $m! \times 2^{m-1}$. The total cost of also finding optimal routes in each individual cluster is therefore*

$$m! \times 2^{m-1} + \frac{1}{2}(n_1! + \dots + n_m!).$$

Proof. Suppose first that an undirected route with two boundary nodes has been found for each cluster. We determine the computational complexity of finding the best global route going through all clusters along these routes using a combinatorial argument. First, a starting cluster is chosen which can be done in m ways. From that cluster, a global starting point should be determined from the pair of that cluster's boundary nodes, which can be done in 2 ways. Next, a second cluster is chosen which can be done in $m-1$ ways. From that cluster, a starting point connecting with the first cluster's end point should be determined from the second cluster's pair of boundary nodes, which can be done in 2 ways. Continuing this way we see that there are $m! \times 2^m$ ways to choose a global route with a start and end. Hence, there are half as many global undirected routes, namely $m! \times 2^{m-1}$.

Now, for a cluster with n_j picking stops there are $\frac{1}{2}n_j!$ ways to choose an undirected route. This is independent of the other clusters. Adding these to the cost of choosing a global route completes the proof. $\square$

The reduction in computational cost described by the theorem is largest when $n_j \sim (n/m)$ for $j = 1, \dots, m$. To give the reader an idea of the magnitude of the reduction in this case, consider a modest $n = 12$. Then $12! \approx 5 \times 10^9$, while if $n_j = n/m$ for $j = 1, \dots, m$ then

$$m! \times 2^{m-1} + \frac{1}{2}(n_1! + \dots + n_m!) = \begin{cases} 724 & m = 2, \\ 60 & m = 3, \\ 204 & m = 4. \end{cases}$$

As can be seen, this approach significantly reduces the computational complexity. Although it does not guarantee an optimal route, it ensures that a route close to the optimal one can be achieved within a reasonable computational time.

The case when $n = 15$ and $m = 3$ is shown in Figure 2. Then $15! \approx 10^{12}$, while $n_1 = n_2 = n_3 = 5$ and

$$m! \times 2^{m-1} + \frac{1}{2}(n_1! + \dots + n_m!) = 204.$$

Of course, we have been comparing the results of the theorem above to a brute force approach for finding optimal routes without employing sophisticated algorithms to reduce computational complexity. However, using for example the Held & Karp algorithm [21] reduces the time complexity from $\mathcal{O}(n!)$ to $\mathcal{O}(n^2 \cdot 2^n)$, which, with $n = 12$, is still 3 orders

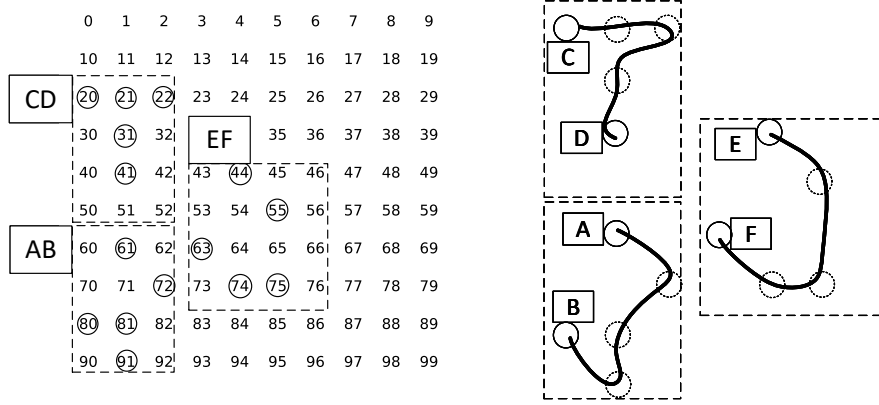


FIGURE 2. An order with 15 stopping positions clustered into 3 regions (left), and the reduced routes for each cluster represented by two boundary nodes (right). For ease of presentation, stopping nodes are labeled from 0 to 99, where node 0 corresponds to $(i, j) = (1, 1)$ in Figure 1.

of magnitude higher than the examples above—and this without using a corresponding algorithm to find optimal routes within clusters. Hence, the gains in time complexity from the approximate method are still substantial.

4. NUMERICAL EXPERIMENTS

Here we perform numerical experiments to evaluate the performance of the resorting algorithm for a warehouse model described in Section 2 with the following parameter settings:

- (1) The warehouse contains $10 \times 10 \times 10$ picking nodes located at $(i, j, k) \in [1, 10] \times [1, 10] \times [1, 10]$. This means that $n_x = n_y = n_z = 10$ in (1).
- (2) The number of article types is $N = 890$, and we assume that the maximum number of articles at a node is 10 for all article types, so $M_{a_{ijk}} = 10$ for all (i, j, k) . Thus, the state space X is

$$X = \{(a_{ijk}, m_{ijk})_{ijk} : a_{ijk} \in [0, 890], m_{ijk} \in [0, 10]\}.$$

- (3) The initial warehouse state x_0 is randomized with the following constraints: It contains 11 xy -coordinate pairs $(i_0, j_0), (i_1, j_1), \dots, (i_{10}, j_{10})$ such that the nodes $(i_0, j_0, k), \dots, (i_{10}, j_{10}, k)$ are empty for all $k \in [1, 10]$. This corresponds to 11 empty racks, for a total of 110 empty nodes. A unique article type $a_{ijk} \in [1, 890]$ is assigned to each of the remaining 890 nodes, and we assume that the initial warehouse state is fully stocked, so the balance at each nonempty node 10.
- (4) An order o to be picked is a collection of 20 individual customer or purchase orders. A purchase order consists of 10 article types with between 1 and 10 parcels of each type. The 20 purchase orders are mutually disjoint, so o consists of 200 article types. In each picking iteration, the 20 purchase orders are divided into three groups using K -means clustering based on where the articles are stored in the warehouse at that time (with $K = 3$ reflecting the classical ABC demand classification scheme, as

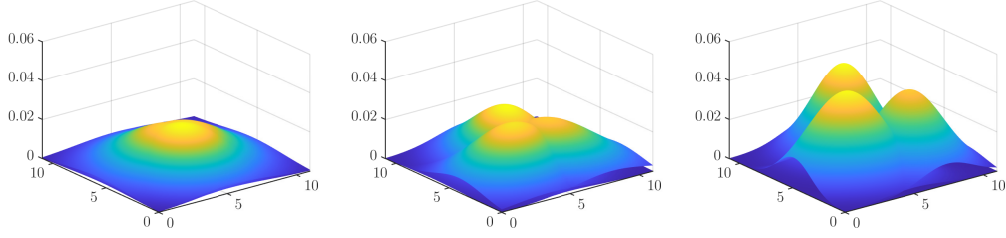


FIGURE 3. Three snapshots of cluster distributions from experiment 1, illustrated by bivariate Gaussian probability density functions defined using each cluster's mean and covariance matrix $\Sigma_n(\ell)$, $\ell = 1, 2, 3$. Each panel shows a superposition of the corresponding probability density function plots. The snapshots are taken at time n (i.e., in the language of §2.4 they show the clusters computed for the n :th order), with $n = 1$ (left), $n = 10$ (middle), $n = 100$ (right). The orders are the same in every iteration. The figure clearly shows that as time progresses, the separation between cluster centers increases, while the bumps become more articulated due to a decreased variance. A movie showing the evolution of the clusters for $n = 1, \dots, 200$ can be found here: https://play.hb.se/media/clusters_unperturbed/0_3r1qr1w4.

discussed in Section 1.1). Each such group is a cluster. An example can be seen in Figure 1.

4.1. Experiment 1. We first perform picking and resorting when the same order is placed over and over, that is, we let $\omega = (o, o, \dots)$ and compute the warehouse state

$$x_n = F_{\theta^{n-1}\omega} \circ \dots \circ F_\omega(x_0) = f(o, \bullet) \circ \dots \circ f(o, \bullet)(x_0), \quad n \geq 1,$$

obtained after filling the order n times, having restocked articles at potentially new locations as determined by the optimization routine. This is depicted in Figure 1, and the experiment shows that continued application of the picking and resorting procedure leads to more pronounced clusters with better separation. To analyze this effect, we study the time evolution of the mean and variance of the clusters by plotting bivariate Gaussian probability density functions defined using each cluster's mean and covariance matrix $\Sigma_n(\ell)$ as time n increases (cf. §2.4). Figure 3 shows the clustering into three clusters of a randomized first order to be picked from an initial randomized warehouse distribution, represented by probability density functions with covariance matrices $\Sigma_1(1), \Sigma_1(2), \Sigma_1(3)$, together with snapshots of the clusters for the 10:th and 100:th picking iteration of the same order, represented by probability density functions with covariance matrices $\Sigma_n(1), \Sigma_n(2), \Sigma_n(3)$ for $n = 10$ and $n = 100$, respectively. Each panel shows a superposition of the corresponding probability density functions. The figure shows that the clusters become more pronounced with better separation as time progresses.

4.2. Experiment 2. To account for small differences in individual orders, we next consider the random example when $\omega = (o_1, o_2, \dots)$ and the orders o_n are independent identically distributed perturbations of the order o used in the previous experiment. As described in §2.3 we then compute the warehouse state after picking the first n orders via

$$x_n = F_{\theta^{n-1}\omega} \circ \dots \circ F_\omega(x_0) = f(o_n, \bullet) \circ \dots \circ f(o_1, \bullet)(x_0).$$

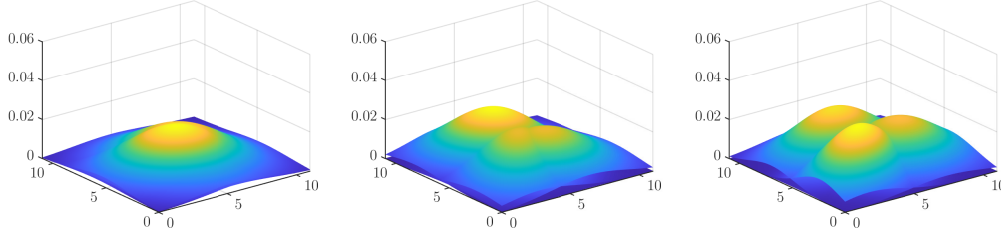


FIGURE 4. Three snapshots of cluster distributions from experiment 2, illustrated by bivariate Gaussian probability density functions defined using each cluster’s mean and covariance matrix $\Sigma_n(\ell)$, $\ell = 1, 2, 3$. Each panel shows a superposition of the corresponding probability density function plots. The snapshots are taken at time $n = 1$ (left), $n = 10$ (middle), $n = 100$ (right). Two picking orders now differ by approximately 10% on average. Although not as pronounced, we still clearly see an increased separation and decreased variance of the clusters as time progresses. A movie showing the evolution of the clusters for $n = 1, \dots, 200$ can be found here: https://play.hb.se/media/clusters_static_perturbation/0_t9n51bme

We let each order o_n be constructed as described under item 4 above, but with the first of the 10 article types in each of the 20 purchase orders randomly selected from the 890 article types with uniform distribution. Clearly, two such purchase orders will differ by approximately 10% in article types, and two consecutive picking orders consisting of 20 such purchase orders that are mutually disjoint to begin with then also differ by approximately 10% in article types, counting multiplicity. The resulting time evolution of clusters when picking and resorting such a sequence of orders $\omega = (o_1, o_2, \dots)$ is described in Figure 4, and we can see that the resorting procedure still leads to clusters becoming more pronounced with better separation, even in the presence of small variations in orders. Note that since the initial clusters depend on both the initial warehouse state x_0 and the first order o_1 , the initial clusters in this experiment will usually be different from the initial clusters in the previous experiment which were obtained from x_0 and the unperturbed order o .

4.3. Experiment 3. For comparison, we perform the same experiment but instead of always replacing the first of the 10 article types in each purchase order with a random article type, we replace a randomly selected article type in each purchase order of the original order o with a random article type. Two such purchase orders will then differ on average by around 19% in article types, and two consecutive picking orders consisting of 20 such purchase orders that are mutually disjoint to begin with also differ by approximately 19% in article types on average, counting multiplicity. The result is described in Figure 5 and we can see that although still there, the clusters are now showing less of a tendency of becoming more pronounced with better separation.

4.4. Cluster separation. As experiments 1, 2, and 3 show, the picking and resorting procedure leads in time to increased separation of the initial clusters, although this separation becomes less pronounced as the variability of the orders increases. To quantify this effect we measure the area of the triangle defined by the cluster center points, and see how this varies with time for the three different setups used in experiments 1, 2, and 3. For experiment 1

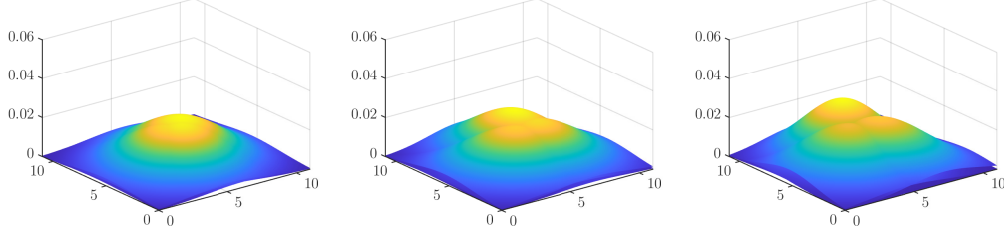


FIGURE 5. Three snapshots of cluster distributions from experiment 3, illustrated by bivariate Gaussian probability density functions defined using each cluster's mean and covariance matrix $\Sigma_n(\ell)$, $\ell = 1, 2, 3$. Each panel shows a superposition of the corresponding probability density function plots. The snapshots are taken at time $n = 1$ (left), $n = 10$ (middle), $n = 100$ (right). Two picking orders now differ by approximately 19% on average, and we only see a slight improvement in the separation and variance of the clusters as time progresses. A movie showing the evolution of the clusters for $n = 1, \dots, 200$ can be found here: https://play.hb.se/media/clusters_random_perturbation/0_g0zx8ooj

the locations of the cluster centers vary fairly slowly, while for experiment 2 and especially 3 the locations vary more rapidly, and as a result the area fluctuates more for experiment 2 and 3. To increase signal strength we therefore average the calculations over 10 runs for each experiment type 1, 2, and 3, and compute the area as a function of time (number of iterations of the picking and resorting algorithm).

The result is presented in Figure 6. For all experiment types, the average area increases from an initial average area of around 0.1; for experiment 1 it has increased to around 6 after the first 20 iterations; for experiment 2 it has increased to around 3 after the first 20 iterations; for experiment 3 it has increased to around 1 after the first 20 iterations. In all cases, the increase is immediate and exceeds one order of magnitude, providing quantitative evidence that the resorting procedure effectively improves cluster separation.

4.5. Quantitative analysis of cluster quality. Here we quantify and compare the increase in cluster concentration and separation for the three experiments using silhouette scores to evaluate cluster quality [30]. We first recall the definition of silhouette score, so let i denote a data point assigned to a cluster A . Define

$$a(i) = \frac{1}{|A| - 1} \sum_{\substack{j \in A \\ j \neq i}} d(i, j),$$

as the average dissimilarity between i and all other points in its own cluster A . Here $d(i, j)$ is the distance between the data points i and j . For every other cluster $C \neq A$, compute the average dissimilarity of i to all points in C , and let

$$b(i) = \min_{C \neq A} \frac{1}{|C|} \sum_{j \in C} d(i, j).$$

The silhouette value for point i is then defined as

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad -1 \leq s(i) \leq 1.$$

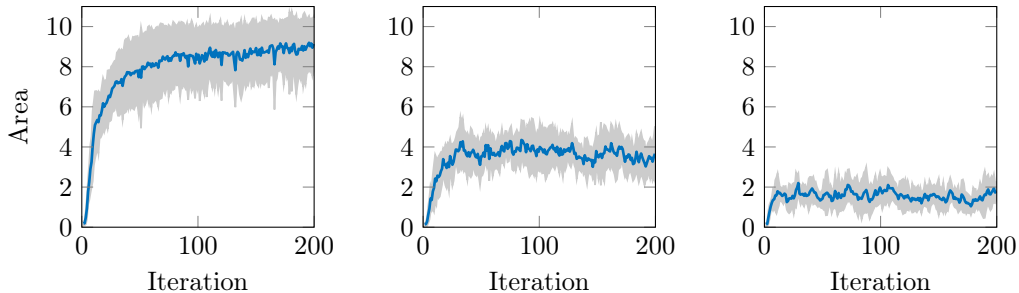


FIGURE 6. Area of the triangle defined by the cluster center points as a function of the number of picking iterations (time) for experiment 1 (left), experiment 2 (middle), and experiment 3 (right). To increase signal strength, the area is averaged over 10 experiments of each type, shown in blue. The gray region indicates the average area plus minus one standard deviation. The average area for the initial warehouse state is around 0.1, and in all three panels this increases substantially within the first 10 or 20 iterations as the clusters become more separated due to the picking and resorting procedure. However, the increase is visibly smaller for experiment 2 and even more so for experiment 3, indicating less pronounced separation of clusters as the noise level increases.

The overall silhouette score of a clustering solution with n points is obtained by averaging over all points:

$$S = \frac{1}{n} \sum_{i=1}^n s(i),$$

with values ranging from -1 to $+1$. Higher scores indicate more compact and well-separated clusters, while values near 0 indicate overlapping clusters and negative values suggest misclassification. As a rule of thumb, scores above about 0.5 reflect strong structure, scores around 0.25–0.5 indicate moderate clustering, and values close to 0 or below denote weak or absent cluster structure.

As in the previous subsection we perform 10 runs for each experiment type 1, 2, and 3 (abbreviated Exp 1, Exp 2, Exp 3 below), and calculate the silhouette score at each iteration. Figure 7 shows the evolution of silhouette scores over 100 iterations for the three experiment types. Exp 1 increases rapidly and stabilizes around 0.45, Exp 2 reaches about 0.10, and Exp 3 increases to around zero. The confidence intervals separate clearly between the experiments.

To quantify these improvements, we compare final to initial silhouette scores within each run. Paired t -based confidence intervals indicate substantial improvement in Exp 1 ($\Delta = 0.47$, 95% CI [0.41, 0.53]), smaller but positive improvement in Exp 2 ($\Delta = 0.12$, 95% CI [0.09, 0.16]), and only marginal improvement in Exp 3 ($\Delta = 0.06$, 95% CI [0.03, 0.08]). These results suggest that, for the parameter settings described in the beginning of Section 4, the method remains effective under low and, to some extent, moderate noise, but yields limited gains under high noise. This is in accordance with the visual evidence of Figures 3–6. We note that under these settings the pick list already covers a substantial fraction of warehouse article types, and this fraction grows further with added noise. (As we will see,

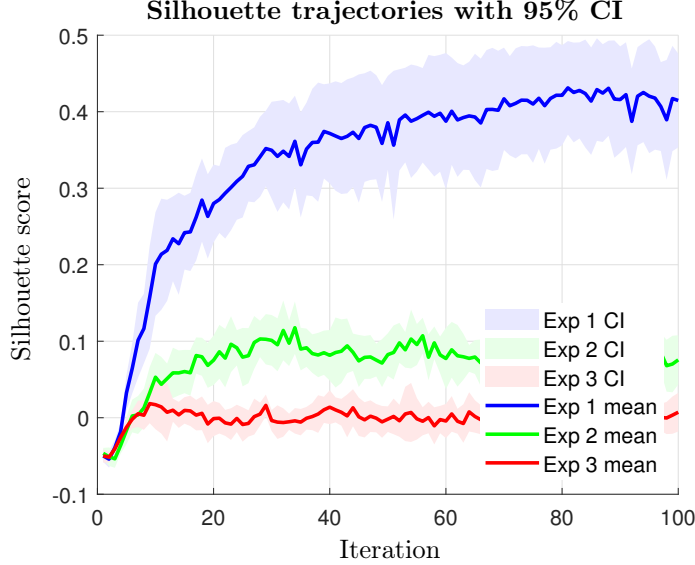


FIGURE 7. Silhouette score trajectories over 100 iterations for a warehouse with $10 \times 10 \times 10$ nodes. Solid lines show the mean silhouette score across runs, with shaded bands indicating 95% confidence intervals. Experiment 1 shows strong and stable improvement, experiment 2 reaches modest gains, and experiment 3 remains close to zero, indicating limited clustering performance under high noise.

when the ratio of pick-list to warehouse article types is smaller, the method performs well also under moderate to high noise, see Section 5.)

To assess whether the magnitude of improvement differs between experiments, we perform pairwise permutation tests on the run-level improvements (10 000 resamples). All comparisons are statistically significant after Bonferroni correction (Exp 1 vs. Exp 2: $p < 0.0001$; Exp 1 vs. Exp 3: $p < 0.0001$; Exp 2 vs. Exp 3: $p = 0.0063$). Effect sizes are also large: Exp 1 dominates both Exp 2 and Exp 3 (Cohen’s $d \geq 5$, Cliff’s $\delta = 1.00$ in both cases), while Exp 2 also outperforms Exp 3 with a large effect (Cohen’s $d = 1.5$, Cliff’s $\delta = 0.76$). For reference, Cohen’s d values of 0.2, 0.5, and 0.8 are often interpreted as small, medium, and large effects, respectively [10]; for Cliff’s δ , thresholds of 0.147, 0.33, and 0.474 have been suggested for small, medium, and large effects [8, 29]. With only three pairwise comparisons, the Bonferroni correction had minimal impact on the results and did not alter the overall conclusions. Thus, although performance degrades with noise, clustering improvements remain detectable even under moderate noise. The final values and statistical comparisons are summarized in Table 1.

4.6. Route optimization. Here we analyze the effects that the resorting algorithm has on route optimization, and showcase the approximate method of calculating optimal routes that we describe in Section 3, performed over 10 independent runs. Each run starts from an arbitrary customer order, consisting of 10 products with varying quantities. The order is assumed to be recurring (as in Experiment 1 above), and at the initial stage, iteration 1, the articles are randomly distributed throughout the entire domain of the warehouse node

TABLE 1. Silhouette score improvements (Δ) with 95% confidence intervals, and pairwise statistical comparisons between experiments for a warehouse with 1000 nodes. Pairwise p -values are from permutation tests (Bonferroni corrected). Effect sizes are reported as Cohen’s d and Cliff’s δ .

	Exp 1	Exp 2	Exp 3
Mean improvement Δ	0.47	0.12	0.06
95% CI	[0.41, 0.53]	[0.09, 0.16]	[0.03, 0.08]
Pairwise comparisons			
Exp 1 vs Exp 2	$p < 0.0001$, $d = 5.0$, $\delta = 1.0$		
Exp 1 vs Exp 3	$p < 0.0001$, $d = 6.3$, $\delta = 1.0$		
Exp 2 vs Exp 3	$p = 0.0063$, $d = 1.5$, $\delta = 0.76$		

TABLE 2. Products and their quantities. The position of each article in the warehouse is encoded through a value between 0 and 99 to describe its (x, y) coordinate in terms of a flattened 10×10 grid starting from bottom left, together with a value between 0 and 9 to describe its z coordinate.

Product	Quantity
a17_4	2
a36_6	2
a57_8	6
a9_7	6
a90_0	6
a77_8	7
a93_6	5
a38_1	4
a23_2	5
a24_4	4

positions. Since the customer order is clustered, it will trigger a reorganization of clustered products, causing them to relocate closer to the cluster center. As seen above, the clusters tend to become more concentrated with reorganization, suggesting that the picking area within each cluster decreases. At iteration 300 we should therefore see a decrease in the total picking distance for each customer order.

We first present one representative trial, choosing the case with the largest deviation from optimal among the 10 independent runs. In this instance, the obtained route length exceeded the optimal by 25%, which constitutes the worst performance observed under the tested settings. The order for this case is presented in Table 2.

When evaluating all possible route permutations through a graph with n vertices, the total number of unique paths is given by $n!$, assuming each vertex is visited exactly once. For each such permutation, $(n - 1)$ transitions must be evaluated to compute the total route cost, resulting in $n!(n - 1)$ operations in total. In the CUDA C implementation (see Subsection 4.7 for details), each route permutation may be assigned to a separate thread, and within each thread, an internal loop is used to process the transitions between vertices. If V

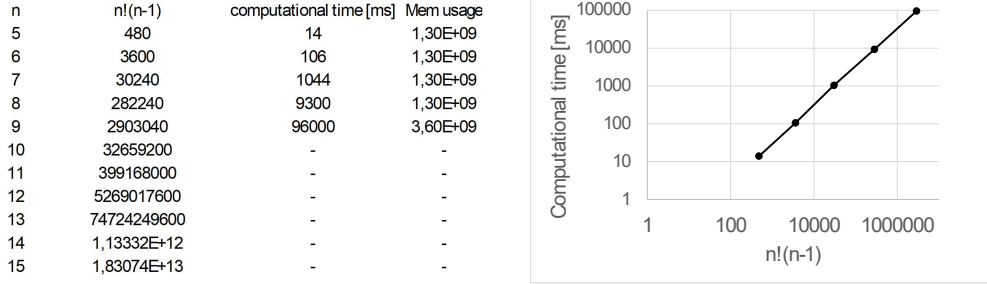


FIGURE 8. Data from the Bellman Ford GPU version (RTX6000, 24GB VRAM) (left), and a comparison between $n!(n-1)$ and computational time (right).

denotes the number of vertices (i.e., the number of stops per route), and $N = n!$ is the total number of permutations, then the computational structure involves operations proportional to $V \times N$. This scaling becomes problematic as n increases, since $n!(n-1)$ grows factorially and dominates memory consumption. Empirically, we observed that values of N larger than 2,903,040 lead to memory errors in the CUDA kernel, likely due to this exponential increase. As a result, processing large values of n without segmenting the computation or distributing it across multiple kernel launches becomes infeasible due to the limitations in available GPU memory, see Figure 8.

Hence, since the number of stop positions in our experiment is $n = 10$, we use the approximate method for calculating an optimal route described in Section 3. Initially, the number of stops is 10 while the number of passed nodes is 27, and the order is picked over a large portion of the warehouse area. By iteration 300, the clustered order regions have shrunk, and the number of passed nodes is now 15, determined using the approximate method. This corresponds to a 44% reduction in route length at iteration 300. At this stage, the reorganization has resulted in many articles being located at the same stopping node, reducing the number of combinations to 6!. This reduction enables a full combinatorial calculation of the optimal route. The result of the simulation is shown in Figure 9. The approximate method does not produce the exact same result as the optimal method, with an optimal distance of 12. However, given the challenges with large $n!$, the approximate method allows for results close to the optimal solution in real-time when using GPUs.

While this example illustrates the worst-performing case, we next summarize results across all runs described in Table 3. The ratio of obtained to optimal route length had a mean of 1.08 (SD = 0.08, median = 1.09), ranging from 1.00 (exact optimal) to 1.25 (worst case). The 95% confidence interval for the mean ratio was [1.02, 1.14], indicating that the method typically produced solutions within 2–14% of the optimal route length despite requiring much less computation. No significant correlation was found between optimal route length and approximation ratio ($r = 0.01$, $p = 0.99$), indicating that performance was stable across different route lengths.

4.7. GPU Implementation. The Bellman–Ford algorithm was parallelized and implemented in CUDA C, targeting an NVIDIA RTX 6000 GPU (Turing architecture, 24 GB GDDR6). Each edge relaxation was assigned to one GPU thread, enabling full parallel

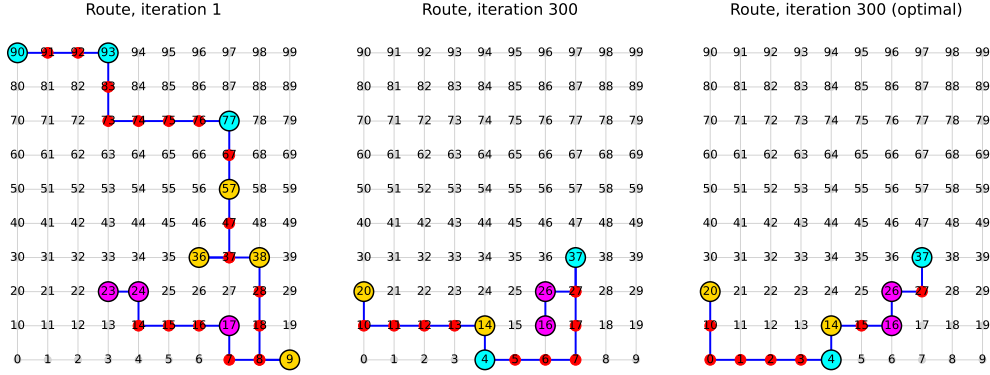


FIGURE 9. Illustrative example of the clustering-based route optimization, showing picking routes for the customer order from Table 2 at different iterations of the resorting algorithm. This is the trial with the largest deviation from optimal among the 10 runs considered, where the obtained route length exceeds the optimal by 25% at final iteration. The stopping positions along the route are indicated by the larger nodes, and they are clustered into 3 color coded regions using the methodology from Section 3. At iteration 1 there are 10 stopping nodes, and the left panel shows the computed route. At iteration 300, the resorting algorithm has reduced the number of stopping positions to 6, and the middle panel shows the computed approximately optimal route with length 15. For comparison, the right panel shows the optimal route after 300 iterations, with length 12.

TABLE 3. Comparison between optimal route length and clustering-based route length across 10 selected cases. Diff = difference between obtained and optimal length. Ratio = obtained/optimal.

Case	Optimal	Obtained	Diff	Ratio
1	13	15	2	1.15
2	11	11	0	1.00
3	8	9	1	1.12
4	7	7	0	1.00
5	11	12	1	1.09
6	12	15	3	1.25
7	14	14	0	1.00
8	14	14	0	1.00
9	10	11	1	1.10
10	12	13	1	1.08
Mean	—	—	—	1.08
SD	—	—	—	0.08
95% CI	—	—	—	[1.02, 1.14]

coverage of all E edges per iteration. The kernel was configured with 256 threads per block, resulting in $\lceil E/256 \rceil$ blocks per launch. For a representative instance with $V = 9716$ vertices and $E = 26\,999$ directed edges, this configuration corresponds to 106 thread blocks per iteration. The graph data were stored in edge-list format using integer arrays $\mathbf{u}$, $\mathbf{v}$, and $\mathbf{w}$ for source vertices, destination vertices, and edge weights, respectively. The distance and predecessor arrays ($\mathbf{dist}$, $\mathbf{pred}$) were allocated in global memory as contiguous vectors of size $V \times N$, where N denotes the number of source vertices processed simultaneously. This layout allows coalesced memory access and efficient indexing across threads. The total device memory footprint for all arrays was approximately 1 MB, which is negligible relative to the available VRAM, confirming that capacity was not a limiting factor.

Although the memory demand is small, performance is constrained by global memory bandwidth. Each thread performs repeated read–write operations to the shared distance arrays across all N sources in every relaxation step. As N increases, these concurrent accesses lead to high write contention and non-coalesced access patterns, saturating the GPU’s memory bus rather than its computational resources. This bottleneck is intrinsic to the Bellman–Ford algorithm when expressed in its fully parallel form: the very feature that enables massive edge-level concurrency also enforces global synchronization after every iteration. Any attempt to restructure the CUDA kernel to alleviate this effect would inherently reduce the degree of parallelism, negating the algorithm’s primary performance advantage. Execution times were measured using CUDA event timers. For the configuration above, the complete GPU execution—including $(V - 1)$ relaxation rounds and host synchronization—completed in approximately 100 ms, corresponding to a throughput of 2627 million traversed edges per second (MTEPS). Profiling confirmed that the kernel is memory-bound, with arithmetic utilization remaining below 20% of theoretical peak. All implementation details and datasets used for validation are available for reproducibility at <https://github.com/mnbe1973/Bellman-Ford-Cuda-C>.

Remark. For validation, we verified that the CPU implementations of Dijkstra’s and Bellman–Ford algorithms produced identical shortest paths on representative test graphs before porting Bellman–Ford to CUDA C. This confirmed functional equivalence between the two approaches. Dijkstra’s algorithm, however, is not straightforward to express in a fully parallel GPU form due to its priority-queue dependence, whereas Bellman–Ford can relax all edges concurrently. Hence, the latter was chosen for GPU acceleration. Meta-heuristic approaches such as ant colony optimization (ACO) were deemed outside the scope of this study. Preliminary CPU–GPU comparisons indicated numerical consistency, while performance gains align with previously reported GPU speedups for edge-relaxation-based algorithms.

5. LARGE-SCALE EXPERIMENTS

To assess the scalability of the proposed method, we perform an additional set of experiments on a warehouse configuration of size $100 \times 100 \times 10$, corresponding to 100 000 nodes. The parameter settings otherwise follow those of Section 4: each picking order consists of a group of 20 customer orders, each with 10 article types and 1–10 parcels of every such article type. This design preserves comparability with Experiments 1–3 above, while allowing us to examine performance in a substantially larger fulfillment center. In this section we thus have the following parameter settings:

- (1) The warehouse contains $100 \times 100 \times 10$ picking nodes located at $(i, j, k) \in [1, 100] \times [1, 100] \times [1, 10]$. This means that $n_x = n_y = 100$ and $n_z = 10$ in (1).

- (2) The number of article types is $N = 89\,000$, and we assume that the maximum number of articles at a node is 10 for all article types, so $M_{a_{ijk}} = 10$ for all (i, j, k) . Thus, the state space X is

$$X = \{(a_{ijk}, m_{ijk})_{ijk} : a_{ijk} \in [0, 89\,000], m_{ijk} \in [0, 10]\}.$$

- (3) The initial warehouse state x_0 is randomized with the following constraints: It contains 1100 xy -coordinate pairs (i_l, j_l) , $0 \leq l \leq 1099$, such that the nodes $(i_0, j_0, k), \dots, (i_{1099}, j_{1099}, k)$ are empty for all $k \in [1, 10]$. This corresponds to 1100 empty racks, for a total of 11 000 empty nodes. A unique article type $a_{ijk} \in [1, 89\,000]$ is assigned to each of the remaining 89 000 nodes, and we assume that the initial warehouse state is fully stocked, so the balance at each nonempty node 10.
- (4) An order o to be picked is a collection of 20 individual customer or purchase orders. A purchase order consists of 10 article types with between 1 and 10 parcels of each type. The 20 purchase orders are mutually disjoint, so o consists of 200 article types. In each picking iteration, the 20 purchase orders are divided into three groups using K -means clustering based on where the articles are stored in the warehouse at that time (with $K = 3$ reflecting the classical ABC demand classification scheme, as discussed in Section 1.1). Each such group is a cluster.

An important consequence of this scaling is that the ratio of picked article types to the total number of available article types is now much smaller than in the previous $10 \times 10 \times 10$ setup. This provides a test of how clustering performs when orders occupy only a small fraction of the storage capacity. Under these conditions, we observed that the method performed well across all noise levels, including Experiment 3, which previously showed only marginal improvement in the smaller warehouse setting.

Due to the larger problem size, we restrict attention in this section to quantitative evaluation using silhouette scores and corresponding statistical validation. In contrast to Section 4, we do not include visual illustrations of cluster compactness or separation, but instead report results directly through statistical measures of cluster quality. Justification for the choice of $100 \times 100 \times 10$ nodes is provided below.

Figure 10 and Table 4 summarize the results for the large warehouse configuration. Substantial improvements in silhouette score were observed in all experiments, with the largest gains in Exp 1 ($\Delta = 0.86$, 95% CI [0.81, 0.90]), followed by Exp 2 ($\Delta = 0.50$, 95% CI [0.45, 0.54]) and Exp 3 ($\Delta = 0.18$, 95% CI [0.15, 0.21]). Pairwise permutation tests confirmed that all differences between experiments were highly significant ($p < 0.0001$ after Bonferroni correction). Effect sizes were uniformly large (Cohen's $d > 5.5$, Cliff's $\delta = 1.0$), indicating clear separation between noise levels. These results show that the clustering method scales effectively to a warehouse with 100 000 nodes and maintains stable performance even under the highest tested noise setting.

In academic benchmarking, Duque-Jaramillo et al. [16] document a distribution warehouse with two sections, 24 rows per section, 66 columns per row and seven levels per column, giving a total of 22 176 storage slots and overall dimensions of 72 m $\times$ 190 m. This is approximately 22% of the 100 000 nodes scale in our large instance and about 22 times the 1000 node scale in our small instance. Alqahtani [2] analyses a retail distribution centre handling 40 SKUs (32 in detailed analysis) within a mixed block-storage and selective-rack layout, equivalent to roughly 0.04% of our large instance and 4% of our small instance.

In industry benchmarking, Digi-Key's Product Distribution Center expansion (PDCe) in Thief River Falls, Minnesota, is a 2.2 million ft² ($\approx 204\,387$ m²) facility—one of the ten largest warehouses in North America—designed to enable nearly triple its previous daily

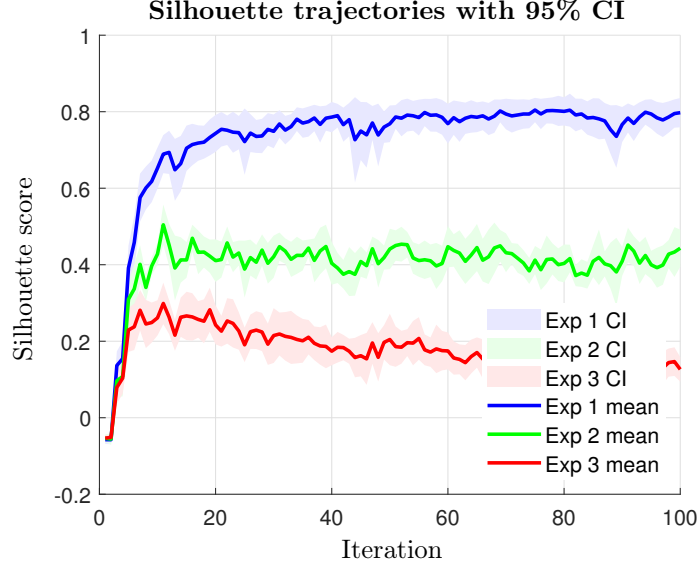


FIGURE 10. Silhouette score trajectories over 100 iterations for the large warehouse configuration ($100 \times 100 \times 10$ nodes). Solid lines show the mean silhouette score across runs, with shaded bands indicating 95% confidence intervals. All experiments show substantial improvements, with higher noise (Exp 3) leading to smaller gains compared to Exp 1 and Exp 2.

TABLE 4. Silhouette score improvements (Δ) with 95% confidence intervals, and pairwise statistical comparisons between experiments for a large warehouse with 100 000 nodes. Pairwise p -values are from permutation tests (Bonferroni corrected). Effect sizes are reported as Cohen’s d and Cliff’s δ .

	Exp 1	Exp 2	Exp 3
Mean improvement Δ	0.86	0.50	0.18
95% CI	[0.81, 0.90]	[0.45, 0.54]	[0.15, 0.21]
Pairwise comparisons			
Exp 1 vs Exp 2	$p < 0.0001$, $d = 5.6$, $\delta = 1.0$		
Exp 1 vs Exp 3	$p < 0.0001$, $d = 12.2$, $\delta = 1.0$		
Exp 2 vs Exp 3	$p < 0.0001$, $d = 5.6$, $\delta = 1.0$		

average of 27 000 packages, with dual conveyor systems spanning more than 27 miles [13, 14]. While facility-level SKU counts are not public, its throughput capacity of approximately 81 000 packages/day allows for a conservative order-of-magnitude estimate. Assuming an average of 5–7 SKUs per package, typical for electronics distributors, this corresponds to roughly 486 000 SKU-picks per day. Although SKU-picks/day is not identical to unique SKUs stored, this approximate throughput represents about five times the SKU-level scale of our large-instance simulation.

These comparisons place our large-instance simulation within the lower order of magnitude of SKU or picking scales observed in major operational warehouses, while also highlighting heterogeneity in SKU density, throughput, and layout. Detailed SKU-level and throughput data for the largest operators, including Amazon, are not publicly disclosed at the facility level due to commercial confidentiality, a limitation also noted by Zhuang et al. [36]. Taken together, these benchmarks indicate that the scale of our simulated instance is representative of modern fulfillment operations while remaining computationally tractable for GPU-based optimization.

6. DISCUSSION AND CONCLUSIONS

The results presented above demonstrate that the proposed clustering-RDS framework can reorganize warehouse layouts dynamically, yielding substantial improvements in route efficiency and cluster compactness. While the experiments rely on synthetic but representative warehouse configurations, the observed effects are systematic and robust across repetitions. Although the simulation setting abstracts from several operational complexities, it captures the essential structural dynamics of storage-location assignment and product reorganization.

The following discussion outlines the robustness and limitations of the proposed framework, evaluates its practical relevance, and identifies directions for future research and industrial deployment.

The implementation of the Bellman-Ford algorithm in CUDA C demonstrates that GPU-based parallelism is highly effective for solving routing problems, particularly when evaluating large numbers of path combinations. While the exponential growth of permutations presents a challenge, especially as the number of nodes increases, this limitation can be mitigated by dividing large routing problems into smaller sub-routes. By processing these segments independently, it is possible to maintain high computational efficiency without exceeding the GPU’s memory or thread capacity. This segmentation strategy ensures that GPU acceleration remains a scalable and practical solution even for complex graph traversal problems. Therefore, the combination of Bellman-Ford and CUDA C provides a flexible framework that balances algorithmic precision with hardware-aware execution.

The numerical experiments demonstrate that the proposed clustering-RDS framework remains robust under a wide range of order variability and warehouse dynamics. In smaller-scale experiments, performance declines as stochastic noise increases, yet clustering effects remain statistically significant and clearly measurable through both geometric and silhouette-based indicators. In the large-scale setting with 100 000 nodes, the method exhibits strong improvements in cluster compactness and separation across all noise levels, showing that its core mechanisms scale effectively even when orders occupy only a small fraction of total capacity. This robustness arises from the adaptive interaction between clustering and relocation: as orders fluctuate, the RDS formulation continuously rebalances the spatial distribution of products to maintain compact storage regions. Such behavior mirrors the adaptive, self-organizing tendencies of modern fulfillment systems, in which storage assignments evolve dynamically in response to changing demand.

While the framework captures key structural dynamics of warehouse reorganization, it relies on several simplifying assumptions that delimit its immediate operational realism. In particular, the model currently assumes single-SKU storage locations (one article type per node), and that articles may be relocated freely at each iteration. These abstractions allow the effects of clustering and route optimization to be studied in isolation, providing a clear baseline for understanding how spatial reorganization improves efficiency. In practice,

relocations are constrained by available labour and may involve non-negligible delays. Such constraints can be incorporated in at least two ways: by placing explicit limits on the number of relocations per iteration (labour capacity constraints), or by assigning a relocation cost or delay to each move (time-based constraints). As discussed in Section 2.1, the mathematical formulation can be extended to incorporate multi-SKU nodes, which together with extensions such as relocation costs and time-based capacity limits represent a natural next step toward operational deployment of the model in real fulfilment environments.

In addition, replenishment—the restocking of depleted picking locations from reserve storage—is implicitly represented in our model through the relocation mechanism itself: when inventory of a given article type at a node is exhausted, the model replenishes it dynamically by assigning a new storage location based on the current clustering configuration. This can be interpreted as a form of adaptive, demand-driven replenishment rather than a static restocking process. Alternatively, replenishment operations could be modeled explicitly as delayed or capacity-limited relocation events, which would constitute natural extensions of the present model and could be integrated with our clustering framework in future work.

Another direction of future research concerns energy-aware warehouse operations, where sustainable storage assignment and system design are increasingly studied in the context of automated systems [28]; such considerations could in principle be combined with clustering-based reorganization to jointly optimize efficiency and sustainability.

Comparable principles are already applied in industry. For instance, Amazon’s robotic picking algorithms dynamically re-cluster storage units based on changing order patterns [1], while Alibaba’s GPU-accelerated routing systems achieve real-time optimization at large scale [22]. These examples demonstrate that the computational and organizational mechanisms analyzed here align with successful industrial practices, supporting the practical viability of the proposed framework.

Future research should also extend the framework toward operational deployment by embedding labour and equipment constraints directly into the RDS model through capacity or cost terms, enabling realistic scheduling of human or robotic pickers. Further, coupling the model with digital-twin or WMS simulation platforms would facilitate real-time decision support, enabling policy evaluation and benchmarking. Empirical validation on real warehouse datasets—beyond the synthetic but representative instances studied here—would establish quantitative benchmarks for cost savings and throughput gains, paving the way for practical adoption in large-scale industrial settings.

Managerial implications and insights. From a practical standpoint, these findings translate into several actionable insights for warehouse managers. The proposed framework serves as a strategic decision-support tool for warehouse planners, logistics engineers, and managers responsible for optimizing storage layouts and order-picking efficiency. By integrating dynamic clustering with route optimization, it provides quantitative insight into how reorganizing stock locations can improve picking performance as demand patterns evolve. The model can be embedded into a WMS platform or used in an offline planning environment to evaluate alternative zoning and replenishment strategies before physical implementation, thereby identifying configurations that minimize travel distances and enhance space utilization.

The numerical experiments demonstrate that the clustering-based reorganization substantially improves order consolidation and reduces average route length, with the approximate GPU-accelerated routing procedure yielding paths within 2–14% of the optimal solution

while maintaining sub-second runtimes for problem sizes up to 10^5 nodes. These results indicate that dynamic, data-driven reallocation of stock locations—implemented even periodically rather than continuously—can produce meaningful operational gains without major infrastructure changes. Compared to traditional static ABC zoning, the proposed method adapts automatically to shifting order profiles, maintaining compact and well-separated product clusters even under moderate noise conditions.

From a managerial perspective, the framework’s simplicity and modularity make it well suited for exploratory “what-if” analyses in digital-twin environments or as an analytical layer within existing WMS platforms. Implementation requires only basic positional data and historical order information, making it both accessible and interpretable for practitioners. In practice, further calibration would be needed to incorporate labour availability, relocation costs, and replenishment schedules—elements that can be introduced as constraints or penalty terms in future extensions. Overall, the framework provides a scalable, computationally efficient foundation for developing adaptive and energy-aware storage assignment policies in next-generation smart warehouses. Taken together, these results demonstrate how mathematically grounded models can directly inform data-driven warehouse management, bridging analytical rigor and managerial practice.

APPENDIX A. WMS LOOP

Here we describe the algorithms of the WMS. The graph G has all information about the position of each product, its current inventory balance, as well as empty positions that can be used for re-positioning a product so that it can be situated closer to a cluster center. (In the mathematical language of §2, the graph is thus a state $x \in X$.) The order graph consists of similar setup as the graph G , with the difference that this graph only has the order content (each article number) and the ordered amount of each product. (The order graph is thus an order o .) The WMS loop has one purpose, to enable orders that belong to a cluster to come closer to each other. This will in an ideal operating condition result in areas in the warehouse where only orders belonging to a specific cluster is situated, see Figure 1.

For clarity, Table 5 summarizes the notation used in Algorithms 1–5.

The *Order Processing* algorithm converts an order ID into a string representation and determines the position to test. It then updates the stock based on the order string and increments the pick count in the graph G , see Algorithm 1.

Algorithm 1: Order Processing

Input: order_id, ordergraf

Output: order_string, test_pos

Function ProcessOrder(*order_id*, *ordergraf*):

order_string, *test_pos* $\leftarrow$ *order_to_string*(*order_id*, *ordergraf*)

 UpdateStock(G , *order_string*)

 increment_pick_count(G , *order_string*)

The *Update Stock* algorithm updates the stock in the graph G based on the provided order string. It ensures that the inventory levels are adjusted according to the orders processed, see Algorithm 2.

TABLE 5. Notation used in Algorithms 1–5.

Symbol / Function	Description
G	Warehouse graph containing product positions and balances
$order_id$	Identifier of the current order being processed
$ordergraf$	Order graph containing article numbers and ordered quantities
$order_index$	Total number of orders in the current batch
$n_repetitioner$	Number of repetitions (iterations) of the main loop
$order_string$	Encoded string representation of an order
$test_pos$	Position index used for testing or stock updates
$next_order_string$	Encoded representation of the subsequent order
$empty_nodes$	Set of empty warehouse nodes available for relocation
$blocked_orders$	Set of orders blocked due to insufficient stock
$specific_point$	Cluster center associated with the current order
$distance$	Distance list between empty nodes and cluster center
$distance2$	Distance list between blocked orders and cluster center
$min_distance$	Minimum distance among candidate relocation nodes
<code>ProcessOrder()</code>	Converts an order ID into a string and updates stock levels
<code>UpdateStock()</code>	Updates inventory in G based on the processed order
<code>CheckStock()</code>	Returns True if sufficient stock exists for an order
<code>FindEmptyNodes()</code>	Identifies empty nodes available for article relocation
<code>CalculateDistance()</code>	Computes distances between node sets and a reference point
<code>MoveArticle()</code>	Moves an article from one node to another
<code>CheckStockAndMove()</code>	Combines stock checking and relocation to balance inventory
<code>Main()</code>	Main WMS loop coordinating order processing and relocation

Algorithm 2: Update Stock

Input: G , $order_string$
Output: None
Function `UpdateStock( $G, order\_string$ ):`
 | `update_stock_from_order( $G, order\_string$ )`

The *Check Stock* algorithm verifies if there is sufficient stock in the graph G to fulfill the next order string. It returns a Boolean value indicating whether the stock is sufficient, see Algorithm 3.

Algorithm 3: Check Stock

Input: G , $next_order_string$
Output: Boolean
Function `CheckStock( $G, next\_order\_string$ ):`
 | **return** `is_sufficient_stock_for_order( $G, next\_order\_string$ )`

The *Check Stock and Move Articles* algorithm first checks if there is sufficient stock for the next order. If the stock is insufficient, it identifies empty nodes and calculates the distances to determine the optimal movement of articles to balance the stock, see Algorithm 4.

Algorithm 4: Check stock and Move Articles**Input:** G , ordergraf, order_id**Output:** None**Function** CheckStockAndMove(G , ordergraf, order_id):

```

    next_order_string  $\leftarrow$  order_to_string(order_id + 1, ordergraf)
    if CheckStock( $G$ , next_order_string) = False then
        empty_nodes  $\leftarrow$  FindEmptyNodes( $G$ )
        specific_point  $\leftarrow$ 
            ordergraf.nodes[order_id].get('cluster_center', 'Unknown')
        blocked_orders  $\leftarrow$  update_stock_from_order2( $G$ , next_order_string)
        distance  $\leftarrow$  CalculateDistance(empty_nodes, specific_point)
        min_distance  $\leftarrow$  find_min(distance)
        distance2  $\leftarrow$  CalculateDistance(blocked_orders, specific_point)
        for  $i \leftarrow 0$  to length(distance2) - 1 do
            test_pos  $\leftarrow$  distance2[i][1] > min_distance[1]
            if test_pos then
                MoveArticle( $G$ , distance2[i][0], min_distance[0], blocked_orders[i][3])
            end
        end
    end
end

```

The *Main Order Processing Algorithm* coordinates the overall process of handling orders. It iterates through a specified number of repetitions and processes each order by converting it to a string, updating the stock, and checking the stock to move articles as necessary, see Algorithm 5.

Algorithm 5: Main Order Processing Algorithm**Input:** ordergraf, G , order_index, n_repetitioner**Output:** None**Function** Main:

```

    for repetition  $\leftarrow 0$  to n_repetitioner - 1 do
        for order_id  $\leftarrow 1$  to order_index do
            ProcessOrder(order_id, ordergraf)
            if order_id < number_of_orders then
                CheckStockAndMove( $G$ , ordergraf, order_id, order_string)
            end
        end
    end
end

```

REFERENCES

1. Russell Allgor, Tolga Cezik, and Daniel Chen, *Algorithm for robotic picking in Amazon fulfillment centers enables humans and robots to work together effectively*, INFORMS Journal on Applied Analytics **53** (2023), no. 4, 266–282.

2. Ammar Y Alqahtani, *Improving order-picking response time at retail warehouse: a case of sugar company*, SN Applied Sciences **5** (2023), no. 1, 8.
3. Ludwig Arnold, *Random dynamical systems*, Springer Verlag, 2013.
4. Richard Bellman, *On a routing problem*, Quarterly of applied mathematics **16** (1958), no. 1, 87–90.
5. Magnus Bengtsson and Jonas Waidringer, *A proposed method using GPU based SDO to optimize retail warehouses*, NVIDIA GTC, San Jose, March 21–24, 2022.
6. Nils Boysen, René De Koster, and Felix Weidinger, *Warehousing in the e-commerce era: A survey*, European Journal of Operational Research **277** (2019), no. 2, 396–411.
7. Federico Busato and Nicola Bombieri, *An efficient implementation of the Bellman-Ford algorithm for Kepler GPU architectures*, IEEE Transactions on Parallel and Distributed Systems **27** (2015), no. 8, 2222–2233.
8. Norman Cliff, *Dominance statistics: Ordinal analyses to answer ordinal questions.*, Psychological bulletin **114** (1993), no. 3, 494–509.
9. Edward G Coffman Jr, Michael R Garey, and David S Johnson, *Approximation algorithms for bin-packing—an updated survey*, Algorithm design for computer system design, Springer, 1984, pp. 49–106.
10. Jacob Cohen, *Statistical power analysis for the behavioral sciences*, Routledge, 1988.
11. Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein, *Introduction to algorithms*, MIT press, 2022.
12. René De Koster, Tho Le-Duc, and Kees Jan Roodbergen, *Design and control of warehouse order picking: A literature review*, European journal of operational research **182** (2007), no. 2, 481–501.
13. *Digi-key celebrates ribbon cutting of product distribution center expansion*, <https://www.digikey.com/en/news/press-releases/2022/august/digi-key-celebrates-ribbon-cutting-of-product-distribution-center-expansion>, 2022, [Accessed 13 Aug. 2025].
14. *Digi-key reaches one millionth package milestone at new product distribution center*, <https://www.digikey.com/en/news/press-releases/2023/november/digikey-reaches-one-millionth-package-milestone-at-new-product-distribution-center>, 2023, [Accessed 13 Aug. 2025].
15. EW Dijkstra, *A note on two problems in connexion with graphs*, Numerische Mathematik **1** (1959), no. 1, 269–271.
16. Juan C Duque-Jaramillo, Juan M Cogollo-Flórez, Cristian G Gómez-Marín, and Alexander A Correa-Espinal, *Warehouse management optimization using a sorting-based slotting approach*, Journal of Industrial Engineering and Management **17** (2024), no. 1, 133–150.
17. Lester R. Jr. Ford, *Network flow theory*, Paper P-923. Santa Monica, California: RAND Corporation (August 14, 1956).
18. Kannan Govindan, Hamed Soleimani, and Devika Kannan, *Reverse logistics and closed-loop supply chain: A comprehensive review to explore the future*, European journal of operational research **240** (2015), no. 3, 603–626.
19. Kevin R. Gue and Russell D. Meller, *The effects of pick density on order picking areas with narrow aisles*, IIE Transactions **38** (2006), no. 10, 859–868.
20. Gaurav Hajela and Manish Pandey, *Parallel Implementations for Solving Shortest Path Problem using Bellman-Ford*, International Journal of Computer Applications **95** (2014), no. 15, 1–6.
21. Michael Held and Richard M Karp, *A dynamic programming approach to sequencing problems*, Journal of the Society for Industrial and Applied mathematics **10** (1962), no. 1, 196–210.
22. Haoyuan Hu, Ying Zhang, Jiangwen Wei, Yang Zhan, Xinhui Zhang, Shaojian Huang, Guangrui Ma, Yuming Deng, and Siwei Jiang, *Alibaba vehicle routing algorithms enable rapid pick and delivery*, INFORMS Journal on Applied Analytics **52** (2022), no. 1, 27–41.
23. Y. Kifer, *Ergodic theory of random transformations*, Birkhäuser, 1986.
24. Mirosław Kordos, Jan Boryczko, Marcin Blachnik, and Sławomir Golak, *Optimization of warehouse operations with genetic algorithms*, Applied Sciences **10** (2020), no. 14, 4817.
25. Masoud Mirzaei, Nima Zaerpour, and Rene De Koster, *The impact of integrated cluster-based storage allocation on parts-to-picker warehouse performance*, Transportation Research Part E: Logistics and Transportation Review **146** (2021), 102207.
26. Charles G. Petersen and Gerald R. Aase, *A comparison of picking, storage, and routing policies in manual order picking*, International Journal of Production Economics **92** (2004), no. 1, 11–19.

27. Zakka Ugih Rizqi and Shuo-Yan Chou, *Dynamic crane scheduling for green automated warehousing: learning-based simulation-optimization approach*, Flexible Services and Manufacturing Journal (2025), 1–28.
28. Zakka Ugih Rizqi, Shuo-Yan Chou, and Adinda Khairunisa, *Energy harvesting for automated storage and retrieval system with sustainable configuration of storage assignment and input/output point*, Transportation Research Part E: Logistics and Transportation Review **192** (2024), 103781.
29. Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek, *Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen'sd for evaluating group differences on the NSSE and other surveys*, Annual Meeting of the Florida Association of Institutional Research **177** (2006), no. 34.
30. Peter J Rousseeuw, *Silhouettes: a graphical aid to the interpretation and validation of cluster analysis*, Journal of computational and applied mathematics **20** (1987), 53–65.
31. Allyson Silva, Leandro C Coelho, Maryam Darvish, and Jacques Renaud, *Integrating storage location and order picking problems in warehouse planning*, Transportation Research Part E: Logistics and Transportation Review **140** (2020), 102003.
32. Steven H Strogatz, *Nonlinear dynamics and chaos: with applications to physics, biology, chemistry, and engineering*, Chapman and Hall/CRC, 2024.
33. Sander Teck, Tú San Phạm, Louis-Martin Rousseau, and Pieter Vansteenwegen, *Deep reinforcement learning for the real-time inventory rack storage assignment and replenishment problem*, European Journal of Operational Research (2025).
34. Gerhard Wäscher, Heike Haußner, and Holger Schumann, *An improved typology of cutting and packing problems*, European journal of operational research **183** (2007), no. 3, 1109–1130.
35. Mehmet Akif Yerlikaya and Feyzan Arıkan, *A novel framework for production planning and class-based storage location assignment: Multi-criteria classification approach*, Heliyon **10** (2024), no. 18.
36. Yanling Zhuang, Yun Zhou, Elkafi Hassini, Yufei Yuan, and Xiangpei Hu, *Improving order picking efficiency through storage assignment optimization in robotic mobile fulfillment systems*, European Journal of Operational Research **316** (2024), no. 2, 718–732.

(Magnus Bengtsson) DEPARTMENT OF ENGINEERING, UNIVERSITY OF BORÅS, SE-501 90 BORÅS, SWEDEN

Email address: `magnus.bengtsson@hb.se`

(Jens Wittsten) DEPARTMENT OF ENGINEERING, UNIVERSITY OF BORÅS, SE-501 90 BORÅS, SWEDEN

Email address: `jens.wittsten@hb.se`

(Jonas Waidringer) DEPARTMENT OF ENGINEERING, UNIVERSITY OF BORÅS, SE-501 90 BORÅS, SWEDEN

Email address: `jonas.waidringer@hb.se`